

Word Embeddings

Jawwad Shamsi

April 4, 2025

1 Introduction to Word Encodings

We need to represent data in a numerical form which can be feed into the neural network. For images, we have used pixel values. However, we need to represent text in a numerical form.

Word encoding is a broader term. It is the process of representing words in a numerical form.

1.1 One hot Encoding

One-hot encoding is one of the most fundamental technique of representing categorical data into a binary format. Each category is represented by a separate column/row with a 1 indicating its presence and 0s for all the other categories.

Table 1 illustrates an example of three types of animals being represented in a one hot encoding format.

Category			
Lion	1	0	0
Tiger	0	1	0
Fish	0	0	1

Table 1: One hot Encoding.

1.1.1 Distance between the two vectors

Distance between the two vectors can be computed using different methods

1. Cosine Similarity

$$\text{Cosine Similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (1)$$

2. Jaccard Similarity

$$\text{Jaccard Similarity} = J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (2)$$

3. Euclidean Distance

$$d(A, B) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2} \quad (3)$$

4. Manhattan Distance

$$d(A, B) = \sum_{i=1}^n |A_i - B_i| \quad (4)$$

where, A and B are two vectors. The above formulae can be used to compute distance between any two vectors. **Example** Compute Cosine similarity between two vectors $X=(5,9,3,0,2,0,0,2,0,0)$ $Y=(3,0,2,0,1,1,0,1,0,1)$ The similarity should be 0.94

1.1.2 Issues with one hot encodings

Following are a few major issues with one hot encoding:

1. One hot encoding is a sparse set of encoding. It means, that number of columns in a vector is equal to the number of unique words or categories in a dataset
2. There is no semantic relationship. In the above example, the distance between lion and tiger is same as the distance between lion and fish.

1.1.3 Word Embeddings

Word embeddings are dense vector representations of words that capture semantic and syntactic relationships between them. Unlike traditional one-hot encoding, which represents words as sparse and high-dimensional vectors, word embeddings map words into a lower-dimensional continuous space where similar words have similar representations. Let us review why do we need Word Embeddings? :

- Captures Semantic Meaning: Similar words have closer representations in the vector space.
- Reduce Dimensionality: Unlike one-hot encoding, embeddings are low-dimensional.
- Improves Generalization in NLP Tasks: Helps models learn from word relationships.
- Useful in Transfer Learning: Pre-trained embeddings can be used across different NLP tasks.

For instance, consider the following vocabulary of 10,000 unique words.

$V = [a, -apple, -, ball, -, Man, -, Q, -, Woman, -, Zulu]$

For a one hot vector, if Man is 4276 then the 4276 th position will hold one and the other will holds 0s.

$$\mathbf{O}_{4276} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Same will be the case for other words such as Woman, Queen, and Apple.

One-hot vectors treat each words seperately and does not generalize the cross words

Example Consider the two sentences:

- I want a glass of Orange —

- I want a glass of Apple —

For both the sentences, juice is the likely word. However, if we use one-hot encodings then there is no imilarity between One-hot vectors of Apple and Orange.

Any product between any two one-hot vecyors is zero. Euclidean distance between pair of vectors is also same.

So the algorithm does not know the difference between King and Queen.

1.1.4 Featurized Representation: Word Embeddings

Let's assume that we would like to create a 300-dimension feature vector. So we can use a 300 dimensional vector for each word. Table 2 illustrate a sample feature matrix. Here e_{5175} and e_{8183} represents feature vectors for man and woman. The goal is to learn these featurized representation. If learned correctly these can capture similarities. For instance, analogy between boy and girl and king and queen.

	Man ₅₁₇₅	Woman ₈₁₈₃	King	Queen	Apple	Orange
Gender	-1	1	-0.95	0.97	0.0	0.01
Royal	0.01	0.02	0.93	0.95	-0.01	0.0
Age	0.03	0.02	0.7	0.69	0.03	-0.02
Food						
Size						
Cost						

Table 2: Featurized Representaion

Suppose Matrix E is the feature vector.

	a ₁	apple ₉	King	Queen	Orange ₆₂₅₇	Zulu

Table 3: Featurized Matrxix

If one-hot encoding of Orange O_{6257}

$$\mathbf{O}_{6257} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

then $E.O_{6257}$ will yield e_{6257} where e_{6257} is the feature vector of Orange Note that E has 300x10,000 dimension whereas O has 10000x1 dimension

More generally, $E.O_j$ will yield e_j the goal is to learn E

Initialize randomly and learn all parameters of E

A language model (LM) is the use of various statistical and probablistic techniques to determine the probability of a given sequence of words occuring in a sentence

Lets assume we are building an LM.

I want a glass of Orange —

The task is to predict the next word.

The whole process can be illustated as follows.

$I_{4343} \text{ want}_{9665} a_1 \text{ glass}_{3852} \text{ of}_{6163} \text{ orange}_{6257}$

The numbers in the subscript denotes the index in the 10,000 words vocabulary.

$$\begin{bmatrix} I- > O_{4353} - - > E- > e_{4353} \\ \text{want}- > O_{9665} - - > E- > e_{9665} - > \\ a- > O_1 - - > E- > e_1 - > \\ \text{glass}- > O_{3852} - - > E- > e_{3852} - > \\ \text{of}- > O_{6163} - - > E- > - > e_{6163} - > \\ \text{orange}- > O_{6257} - - > E- > e_{6257} - > \\ - > \text{Softmax}_{10000} \end{bmatrix}$$

Since we are dealing with a six-word input, we will need a NN with six inputs. The matrix below indicates different layers of the NN. We can learn

embeddings using a neural network as follows.

1. At the input layer, we have one-hot encodings for each word.
2. At the hidden layer, we have the vector E . The vector takes input from all the inputs. The hidden layer has 10,000 neurons - representing the number of words in the vocabulary.
3. The next hidden layer is vector e which represents the feature vector for each input word. The feature vector has 300 dimensions. For 6 words this implies that it will be a vector of size 1800. The weights of this vector will represent the actual features of the word
4. The output layer has softmax with 10,000 possibilities.

The whole process can also be understood from figure ?? . Note that at the output layer, one-hot encoding is used with softmax.

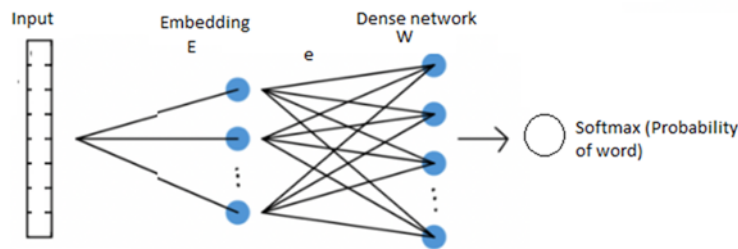


Figure 1: embeddings

Training The final output is a one-hot encoding vector of Orange. So the weights learned during the training process serve as the embedding vector. Note that e is a vector of size 300. So the fully connected layer has 1800 neurons. Predicting the next word is a reasonable way of learning. We use back propagation to learn the weights In order to cater the variable size of sentences, we use a fixed size window.

1.2 Word2Vec

Word2Vec is a popular algorithm to generate word embeddings, developed by researchers at Google. It is based on neural networks and comes in two major variants:

- Continuous Bag of Words (CBOW)
- Skip-gram

1.2.1 Continuous Bag of Words (CBOW)

In this technique the target word is predicted based on the surrounding context words. **Training Process**

1. Uses a sliding window over a corpus.
2. Given a window of words, the model learns to predict the missing center word. • Advantages:
3. Faster to train compared to Skip-gram.
4. Works well for frequent words. • Example:
5. Sentence: The cat sat on the mat.
6. Context: ["The", "sat", "on", "the"] → Target: "cat"

1.2.2 Skip-gram Model

In this technique, context is predicted given a target word. **Training Process**
Given a word, it tries to predict the words surrounding it within a fixed window size.

Instead of predicting one word (like CBOW), it predicts multiple words. • Advantages:

Performs better for rare words.

Captures more complex word relationships. • Example:

Sentence: The cat sat on the mat.

Target: "cat" → Context: ["The", "sat", "on", "the"]

In the Skip-gram model, we train a neural network to predict context words given a target word.

1.2.3 How do we Tokenize or Pre-Process

Let's assume we have the following small dataset.

Deep learning is amazing. Machine learning is powerful.

1. We can tokenize Convert the text into a list of words: corpus = ["Deep", "learning", "is", "amazing", "Machine", "learning", "is", "powerful"]
2. Build the Vocabulary Vocabulary is a set of unique words in the corpus vocab = sorted(set(corpus)) print(vocab) **Output** ['Deep', 'Machine', 'amazing', 'is', 'learning', 'powerful'] Now, we have 6 unique words.
3. Assign Each Word a Unique Index We create two dictionaries:
 - word2idx → Maps each word to a unique index
 - idx2word → Maps each index back to its word

```
word2idx = word: idx for idx, word in enumerate(vocab)
idx2word = idx: word for word, idx in word2idx.items()
```

Resultant Mapping word2idx = 'Deep': 0, 'Machine': 1, 'amazing': 2, 'is': 3, 'learning': 4, 'powerful': 5 idx2word = 0: 'Deep', 1: 'Machine', 2: 'amazing', 3: 'is', 4: 'learning', 5: 'powerful' This mapping allows us to represent words to be represented as indices